Contents

Solver Guide for Practitioners

A plain-language introduction to computational solvers: what they are, when to use them, and which one to pick.

v0.2 — Synthesized from reviews by GPT-4o/o3, Claude Opus 5, Gemini 2.5 Pro, GLM-4-7 (Zhipu AI), and Qwen-3-7-plus (Alibaba).

Solver Advisor Architecture

The diagram below (contributed by GPT-4o) shows how the six-stage NL→solver pipeline is structured. The LLM acts as translator; the solver is the source of truth.

Figure (illustrative, not the authoritative design): GPT-4o's six-stage sketch of an NL→solver pipeline. Useful as a mental map; not rigorous. Key gaps: the 0.72 confidence threshold is arbitrary; the clarify loop and gap-detection path are missing; Stage 3 (LLM formulation) is the highest-risk step (§4.15) but is presented as routine; the final "Certificate" label conflates C1 solver status with C7 kernel-verified proof (§10); C0 model validation and network flow are absent from the solver palette.

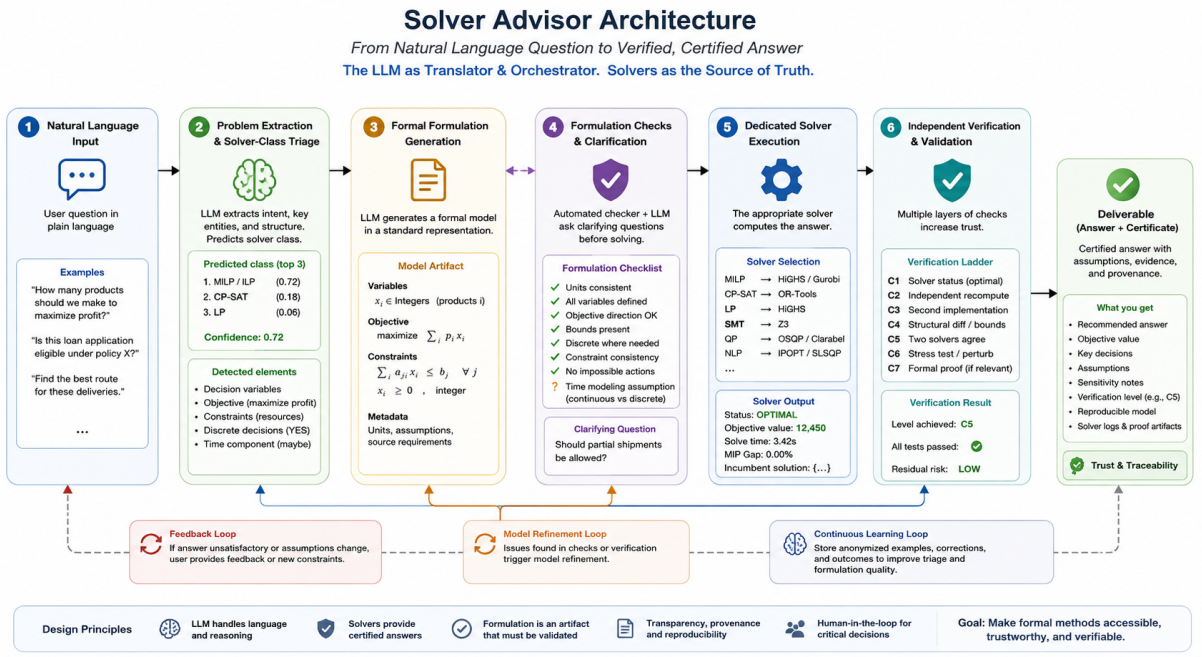


Figure 1: Solver Advisor Architecture

1 What is a Solver?

A **solver** is software that finds a deterministic, verifiable answer to a precisely stated question.

Solvers span a much wider territory than mathematics and optimization. Any time a problem can be formally specified — in rules, constraints, schemas, queries, or equations — a solver can compute and certify the answer.

You already know this from everyday life:

Real-world question	Solver type	Tool
"How do I drive from A to B fastest?"	Graph / shortest-path	networkx (Dijkstra)
"Which customers match these criteria?"	Query / database	SQL (sqlite3, DuckDB)
"Which shifts minimize total overtime cost?"	Optimization (ILP)	PuLP / OR-Tools
"Is this loan application eligible under policy X?"	Logic / rule engine	Z3 (SMT solver)
"Does this dataset have any nulls in required columns?"	Data validation	pandera / Great Expectations

Real-world question	Solver type	Tool
“What portfolio beats the market with least risk?”	Optimization (QP)	cvxpy
“Is this theorem provably true?”	Symbolic / formal proof	SymPy, Lean 4
“Which delivery zones overlap?”	Geometric / spatial	Shapely
“What price should I charge to maximize revenue?”	Optimization (NLP)	scipy.optimize
“Cheapest way to ship goods across a network?”	Network flow	networkx (min-cost flow)

The key word is *precisely*: to use a solver you must translate your question into a formal specification — variables, constraints, rules, a schema, or a query — that the solver understands. That translation — not the computation — is where most of the work happens.

Solvers do not guess. They return a certified answer: optimal, feasible/infeasible, SAT/UNSAT, pass/fail, or a verified proof. Unlike an LLM (probabilistic) response, a solver answer is checkable by definition.

---

2 Optimization Problems: The Three Ingredients

Optimization is one major solver family — it finds the *best* value under constraints. Every optimization problem has exactly three parts:

1. Decision variables — what you can control (x_1, x_2, … x_n)
2. Objective — what you want to optimize (maximize profit, minimize cost)
3. Constraints — rules that all solutions must obey (budget ≤ $10K, hours ≤ 40)

Example — production planning:

> A factory makes chairs and tables. Each chair earns $50 profit; each table earns $80. We have 100 hours of labor and 60 kg of wood. A chair takes 2h labor + 1 kg wood; a table takes 4h + 2 kg. How many of each should we make?

Ingredient	This problem
Decision variables	`chairs` (integer), `tables` (integer)

Ingredient	This problem
Objective	maximize `50·chairs + 80·tables`
Constraints	`2·chairs + 4·tables ≤ 100` (labor), `chairs + 2·tables ≤ 60` (wood), both ≥ 0

This is an **Integer Linear Program (ILP)** — the simplest class where solvers shine.

---

3 Solver Decision Guide — Which One to Use?

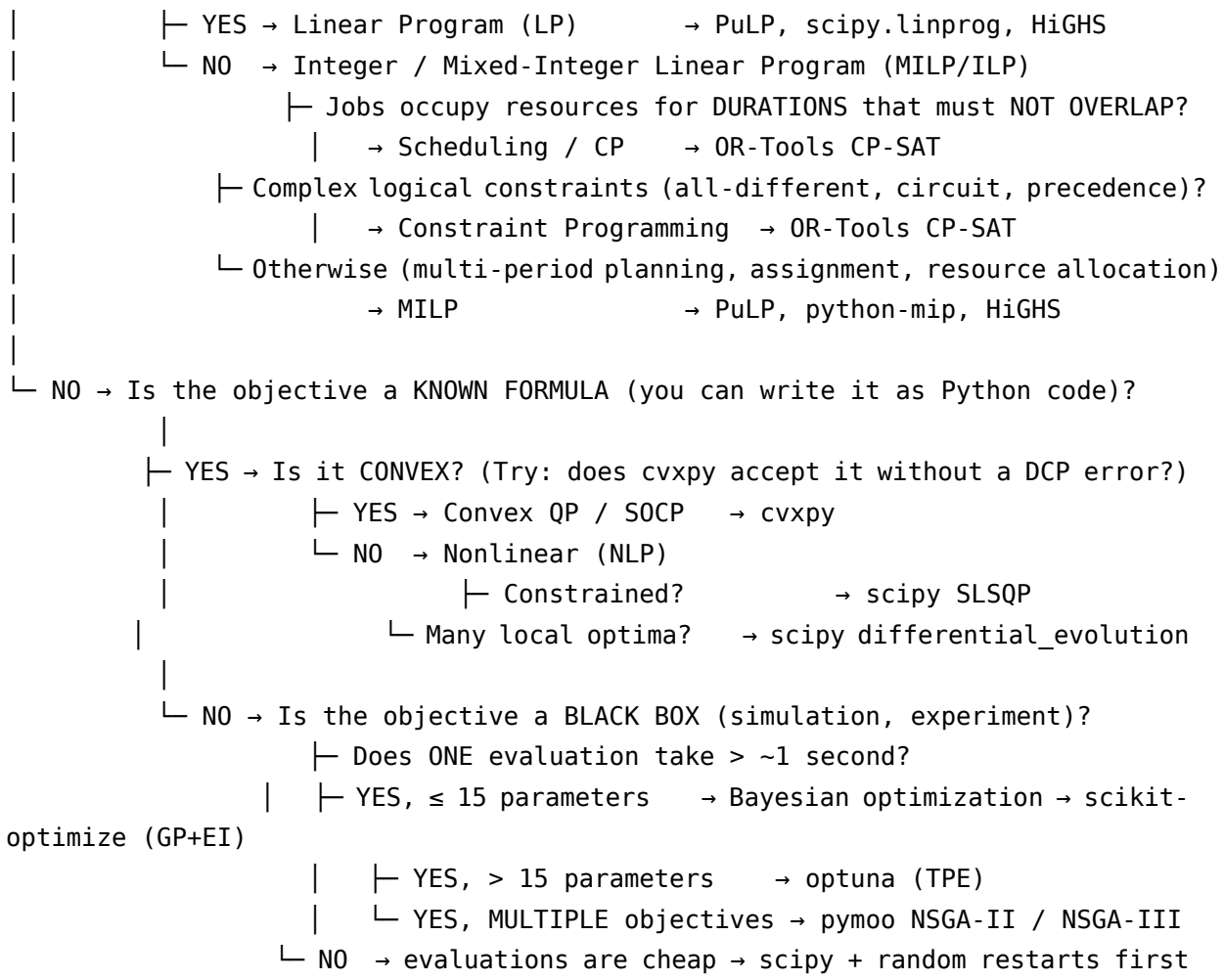
Start at Step 0. If your problem is an optimization problem (find the best value), continue to Step 1.

```
Step 0 — What KIND of answer do you need?

  ├─ OPTIMAL VALUE (maximize profit, minimize cost, find shortest route)
  │    → Continue to Step 1 (optimization flowchart below)
  │
  ├─ YES / NO (does this satisfy rules? is the data valid? is this feasible?)
  │    ├─ Logical rules / policy      → SAT / SMT (Z3)
  │    ├─ Data schema / statistics    → Data validation (pandera, Great Expectations)
  │    └─ Hard combinatorial rules    → CP / SAT (OR-Tools CP-SAT)
  │
  ├─ WHICH RECORDS (filter, aggregate, join structured data)
  │    → SQL / database engine (sqlite3, DuckDB, pandas)
  │
  ├─ SYMBOLIC / FORMAL ANSWER (prove an identity, solve an equation symbolically)
  │    ├─ Algebra / calculus / ODE     → SymPy / SageMath
  │    └─ Kernel-verified formal proof → Lean 4
  │
  └─ SPATIAL ANSWER (overlap, coverage, containment, distance)
       → Geospatial (Shapely, geopandas)


Step 1 — Optimization flowchart (for OPTIMAL VALUE problems):

Is your objective and all constraints LINEAR (no x², x·y, exp(x), …)?
│
├─ YES → Are all decision variables CONTINUOUS (not forced to integers)?
```



> **Important:** "Any time component" does NOT automatically mean CP-SAT. Production planning with monthly time periods, lot-sizing, and inventory balance are textbook MILPs — time is an index, not a scheduling constraint. Use CP-SAT only when jobs physically occupy resources for a duration and must not overlap.

Special cases — not covered by the optimization flowchart:

Problem pattern	Solver class	Library
Must satisfy rules / logic (eligibility, compliance)	SAT / SMT	Z3
Combinatorial feasibility, hard constraints, no objective	Constraint Programming	OR-Tools CP-SAT
Routing / visit all locations (TSP, VRP)	CP routing	OR-Tools routing

Problem pattern	Solver class	Library
Graph problems (shortest path, matching, flow)	Graph / network	networkx
Transportation, assignment, min-cost flow	Network flow (LP, polynomial-time)	networkx, scipy
Multiple rational agents competing	Game theory	nashpy, Gambit
Objective is uncertain / scenario-based	Stochastic LP	mpi-sppy, Pyomo
Dataset must satisfy statistical schema	Data validation	pandera, Great Expectations
Location / geometry (overlap, coverage, containment)	Geospatial	Shapely

> **Network flow tip:** If your MILP constraint matrix has at most one +1 and one −1 per column (transportation, assignment, bipartite matching), it is **totally unimodular (TU)**. The LP relaxation is automatically integer-valued — no branch-and-bound needed. Use `networkx.min_cost_flow()` instead of PuLP: typically 10–100× faster with no integrality gap.

---

4 Solver Classes in Plain Language

4.1 Linear Program (LP)

When: everything is linear, all variables are continuous, one objective.

Intuition: draw the feasible region (a convex polygon), then slide the objective line until it just touches a corner — that corner is the answer.

Properties: - Always finds the global optimum (if it exists) - Scales to millions of variables and constraints - Shadow prices (duals) come for free — see §4.14

Python library: `PuLP`, `scipy.optimize.linprog`, `HiGHS`

```
from pulp import *
prob = LpProblem("chairs_tables", LpMaximize)
chairs = LpVariable("chairs", lowBound=0)
tables = LpVariable("tables", lowBound=0)
prob += 50*chairs + 80*tables
```

```
prob += 2*chairs + 4*tables <= 100     # labor
prob +=   chairs + 2*tables <= 60      # wood
prob.solve(HiGHS_CMD(msg=0))
```

Verify: check `LpStatus[prob.status] == "Optimal"` and plug values back into constraints.

4.2 Integer / Mixed-Integer Linear Program (MILP / ILP)

When: same as LP, but some or all variables must be whole numbers (people, machines, binary on/off).

Intuition: LP solves instantly; forcing integers makes it NP-hard. Modern solvers use *branch-and-bound*: solve LP relaxation, branch on a fractional variable, prune branches that can't beat the current best.

Properties: - Small problems (< 1,000 binary variables) solve in seconds; large instances may need time limits - MILP is the workhorse of supply chain, scheduling, and resource allocation - Fixed costs (pay $5K to open a plant, then $2/unit) require binary variables — LP cannot model this

> **Common mistake:** if your constraint matrix is totally unimodular (transportation, assignment, matching), MILP is unnecessary and slow. Use network flow algorithms instead — polynomial-time and no integrality gap. See M8.

Python library: `PuLP` (use HiGHS backend), `python-mip`, `OR-Tools`

4.3 Constraint Programming (CP)

When: combinatorial problem with no natural linear structure — scheduling with no-overlap, nurse shift exclusions, exam timetabling.

CP vs. MILP decision rule:

Use CP-SAT when	Use MILP when
Jobs occupy resources for durations and must not overlap	Costs and quantities flow between time periods
Constraints are logical (all-different, no-overlap, circuit, precedence)	Constraints are arithmetic inequalities

Use CP-SAT when	Use MILP when
Integer variables are assignments or sequences	Integer variables are discrete quantities
You need any feasible solution quickly	You need a proven near-optimal objective value

Key property: big-M MILP formulations of disjunctive constraints have extremely weak LP relaxations, causing enormous branch-and-bound trees. CP-SAT propagates these constraints directly — often orders of magnitude faster.

Python library: `ortools.sat.python.cp_model`

4.4 Scheduling / Routing

When: assign jobs to machines over time (job-shop) or routes to vehicles (VRP/TSP).

Key metrics: makespan (total completion time), OEE (machine utilization), fill rate (orders fulfilled on time).

Python library: `ortools` CP-SAT (scheduling with interval variables), `ortools` routing (VRP/TSP)

4.5 Convex Quadratic / Semidefinite Programs (QP / SDP)

When: objective is quadratic (e.g. portfolio variance = $x^\top \Sigma x$) and everything is convex.

Practical DCP test: if `cvxpy` accepts your expression without a "DCP rule violation" error, the problem is convex and a global optimum is guaranteed.

Python library: `cvxpy`

```
import cvxpy as cp, numpy as np
w = cp.Variable(n)
prob = cp.Problem(cp.Maximize(mu @ w - gamma * cp.quad_form(w, Sigma)),
                  [cp.sum(w) == 1, w >= 0])
prob.solve()
```

4.6 Nonlinear Optimization (NLP)

When: objective or constraint is nonlinear AND you can write the formula as code.

Regime	Algorithm	Use when
Smooth, constrained	SLSQP (`scipy.minimize`)	Differentiable; one global optimum likely
Non-smooth	Nelder-Mead	No gradient available, low-dimensional
Many local optima	`differential_evolution`	Non-convex; budget allows thousands of evaluations

> **Global optimum warning:** `differential_evolution` is a heuristic — it does not guarantee finding the global optimum. The comparison table labels it "global" meaning *global search strategy*, not *global optimality certificate*. For a certified global NLP optimum you need spatial branch-and-bound (BARON, Couenne).

Python library: `scipy.optimize`

4.7 Multi-objective Optimization (Pareto front)

When: two or more conflicting objectives with no single "best" — only tradeoffs.

Multi-objective vs. many-objective:

	Multi-objective	Many-objective
Objectives	2–3	4+
Algorithm	NSGA-II	NSGA-III (reference directions)
Why different?	Dominance comparison works well	Dominance resistance collapses at 4+ objectives
Visualization	2D/3D Pareto front	Parallel coordinates, radar chart

Why dominance collapses at 4+ objectives (dominance resistance):

Pareto dominance requires solution A to be at least as good as B on *every* objective. For randomly distributed points in M-dimensional space, the probability that any

point dominates another falls sharply as M grows. By 4–5 objectives, essentially every individual is non-dominated — the entire population reaches rank 1 and non-dominated sorting produces no ordering. NSGA-II then relies entirely on crowding distance, which rewards solutions at the extremes of objective space. The population converges, but to a diverse set of dominance-resistant boundary solutions — well-spread but far from the true Pareto front. **Commit to 4 objectives as the NSGA-II reliability limit** (empirical basis: Deb & Jain, 2014).

How NSGA-III fixes this — reference directions:

NSGA-III replaces crowding distance with a set of pre-specified *reference direction vectors*, uniformly distributed on the unit simplex in objective space. During survival selection, each solution is associated with its nearest reference direction. The algorithm ensures each direction has at least one representative — diversity becomes a scheduling constraint rather than an emergent property. This forces the population to cover the interior of the high-dimensional front, not just its boundary extremes.

Visualizing a 5-objective front: - **Parallel coordinates** (primary): each objective is a vertical axis, each solution a polyline. Conflicts appear as crossing lines; redundant objectives appear as parallel bundles (signal to reduce dimensionality). - **Radar chart**: effective for comparing a shortlist of 5–10 solutions, not for showing the full front. - **Practical advice:** filter the front using decision-maker thresholds first (e.g., "cost < $2M and service > 95%"), then visualize what survives. 10,000 raw Pareto points are not actionable.

When MOEA/D is preferable to NSGA-III: when the Pareto front geometry is irregular, disconnected, or degenerate. MOEA/D decomposes the problem into N scalar subproblems cooperatively — better at covering irregular regions. Prefer NSGA-III by default for novel problems.

> **Tip: Exact alternative for linear objectives:** if all objectives are linear in the same decision variables, use **ε-constraint scalarization** over an exact LP/MILP solver. This traces the *true* Pareto front with optimality certificates — no evolutionary heuristic needed.

Python library: `pymoo` (NSGA-II, NSGA-III, MOEA/D)

4.8 Stochastic / Robust Optimization

When: uncertainty in demand, prices, or machine failure that you need a solution robust to across scenarios, not just on average.

Approach	Key idea	Library
Two-stage stochastic LP	Decide now (stage 1); take recourse after observing the scenario (stage 2)	Pyomo / mpi-sppy
Robust MILP	Objective must hold under worst-case uncertainty realization	python-mip + manual robust constraints

> **Note:** "uncertain demand" in a problem description does not automatically mean stochastic programming. Often the right model is deterministic with a safety margin. Only use stochastic programming when you have quantified scenario probabilities and the recourse decision is explicitly modeled.

4.9 Game Theory

When: multiple rational agents optimize their own objectives, and one agent's decision affects the others' payoffs.

Game type	Players move	Solution concept	Library
Normal-form (simultaneous)	At the same time	Nash equilibrium	`nashpy`
Extensive-form (sequential)	In turn	SPNE via backward induction	`openspiel`

When does a solver become necessary? A competent analyst can solve 2×2 games by inspection. A solver becomes genuinely necessary at: **2 players with ≥ 5 strategies each** (mixed-strategy support enumeration is non-trivial by hand), or **≥ 3 players** (`nashpy` supports only 2-player games — use `Gambit` for 3+ players, where finding Nash equilibrium is PPAD-complete).

Key insight: game-theory solvers prove what rational agents *will do*, not what they *should do* cooperatively.

4.10 Black-Box / Bayesian Optimization

When: the objective has no formula — it is a simulation or physical experiment — AND each evaluation is expensive (> ~1 second).

> **If evaluations are cheap** (< 1 second), use `scipy.minimize` with random restarts first. Bayesian optimization's advantage is *sample efficiency* — it only matters when trials are genuinely expensive.

Library	Algorithm	Best for
`scikit-optimize`	GP + EI	≤ 15 parameters, tight budget (≤ 50 trials)
`optuna`	TPE	> 15 parameters, mixed types, budget 50–500 trials
`ax-platform` / `botorch`	BO + multi-fidelity	Production ML tuning, noisy objectives

GP+EI degrades when: parameters exceed 15–20 (switch to TPE or TuRBO); noise is high (model noise explicitly; report posterior-mean best, not best observed); evaluations exceed ~1,000 (GP fitting is $O(n^3)$ — switch to TPE).

4.11 SAT / SMT Solvers

When: problem is expressed as logical rules and you need to know whether a satisfying assignment exists.

Python library: `z3-solver`

```
from z3 import *
x = Int('loan_amount'); y = Int('credit_score')
s = Solver()
s.add(y >= 700, x <= 250000)
print(s.check())   # sat / unsat
```

When SAT/SMT beats LP: if constraints involve logical implications (`IF income > 50K THEN max_loan = 300K ELSE 150K`), Z3 handles this directly; LP cannot.

4.12 Data Validation / Schema Solvers

Library	What it checks	Approach
`pandera`	DataFrame schema (types, ranges, null counts, statistics)	Declarative schema → validation → error report
`great_expectations`	Expectation suites (percentile bounds, referential integrity)	Expectation objects → JSON result

Use case: data pipeline quality gates — run before feeding data to an optimizer to catch upstream errors that would silently produce wrong answers.

4.13 Geospatial Solvers

When: problem involves geographic shapes: service area coverage, delivery zone design, facility placement.

Key operations: `intersection`, `union`, `difference` of polygons; point-in-polygon; buffer zones; Voronoi tessellation; distance matrices (feeds into VRP, §4.4).

Python library: `shapely`, `geopandas`

4.14 Shadow Prices and Sensitivity Analysis

Identified by multiple reviewers as the most important missing concept for practitioners.

Every LP/QP solve gives you two outputs. Most practitioners take one and discard the other.

The first is the **primal solution**: make 20 chairs, open 3 warehouses, assign nurse 7 to OR-2. The second is the **dual solution** — the **shadow price** on each constraint, telling you *how much the objective improves if that constraint is relaxed by one unit*.

Example: the factory from §2. LP says: make 20 chairs and 15 tables, profit = $2,200. The dual says: - Labor shadow price: **$18/hour** (binding — labor is the bottleneck) - Wood shadow price: **$0** (not binding — wood has slack)

This tells you: stop buying extra wood inventory (zero marginal value); overtime up to $18/hour is profitable, above that it is not. You didn't need a second solve — this was in the first solve all along.

Three rules for safe use:

1. **Read sign and slack together.** A non-binding constraint has shadow price zero. If a constraint you *expect* to bind has zero shadow price, your data or model is wrong. This is the cheapest model-validation check available.
2. **Respect the validity range.** A shadow price is a local derivative, valid only over a range (the RHS ranging interval). "Labor is worth $18/hour" does not mean 1,000 extra hours are worth $18,000 — at some point another constraint becomes binding. Re-solve for large changes.
3. **Do not read duals from a MILP.** Integer variables destroy LP duality. The duals from the final LP relaxation node are *not* shadow prices of the integer problem. For MILP sensitivity: perturb the RHS, re-solve, compare.

```
prob.solve()
for name, c in prob.constraints.items():
    print(f"{name:20s}  shadow price {c.pi:8.2f}   slack {c.slack:8.2f}")
# cvxpy: constraint.dual_value
# Pyomo: activate the `dual` Suffix before solving
```

Never ship a solver result to a decision-maker without the shadow prices. The primal tells them what to do this week; the dual tells them what to change.

4.15 Formulation Risk: The Silent Failure

Identified as the most important safety concept by all five reviewers.

A solver can be **perfectly correct and still give the wrong answer.** This is the most important distinction in any solver-assisted workflow.

Computational problem-solving has two layers:

```
1. World → mathematical formulation      ← the dangerous boundary
2. Mathematical formulation → answer     ← what the solver certifies
```

When an LLM or a practitioner produces the formulation, layer 1 is the new failure point.

Example: *"Minimize delivery cost while ensuring every customer receives an order on time."*

A mathematically correct MILP can still be wrong because "on time" was modeled as a hard deadline when it should be a 95% service-level target, or transportation time was confused with calendar time. The solver says "Optimal." The plan is operationally infeasible.

Formulation validation checklist:

Check	Question
Units	Are all quantities in compatible units? (km vs. m, kg vs. tonnes)
Integrality	Are discrete decisions (people, machines, open/closed) declared integer?
Bounds	Is every variable bounded? (unbounded maximization → solver returns "Unbounded")
Objective direction	Are you maximizing profit or minimizing cost — not accidentally the reverse?
Binding constraints	Do the binding constraints (shadow price ≠ 0) make business sense?
Degenerate case	Solve with zero demand / one unit — do you get the intuitively correct answer?
Expert review	Can a domain expert who didn't build the model confirm the constraint list matches the problem?

The key insight: a system that routes NL → correct solver class but generates the *wrong formulation* is more dangerous than one that routes to the wrong class, because the certification ("Optimal") makes the wrong answer more convincing.

---

5 Solver Performance Tuning

Unanimously identified as missing from v0.1.

5.1 MILP: Time Limits, Gaps, and Warm Starts

Always set a time limit and always report the gap.

```
# PuLP + HiGHS (recommended default for all new projects)
prob.solve(HiGHS_CMD(timeLimit=300, gapRel=0.01))
# stops when within 1% of optimal OR after 5 minutes
```

A result of "€1.2M, 0.4% gap" is honest. "€1.2M, Optimal" after hitting a time limit is a lie the API will tell you if you don't check the status. Almost all business value

is captured in the first 1–5% of gap closure.

Warm starts (for re-solves: daily rosters, rolling horizon plans): feed yesterday's solution as the starting incumbent. In PuLP: `var.setInitialValue(v) + warmStart=True`. In CP-SAT: `model.AddHint(var, value)`. Warm starts cut re-solve time 2–10× and produce solution stability — schedules that don't randomly reshuffle between runs, which often matters more than the last 0.5% of objective.

5.2 Presolve and Cutting Planes

Presolve runs before branch-and-bound: removes redundant constraints, fixes variables, tightens coefficients, detects trivial infeasibility. Shrinks models 30–60% and should almost never be disabled (only for debugging).

Cutting planes add valid inequalities during solving that cut off fractional LP solutions without removing integer points (Gomory, cover, clique cuts). Help most on models with weak LP relaxations — precisely the big-M disjunctive models described in M7. If you find yourself increasing cut aggressiveness, the better fix is usually to reformulate with tighter linking constraints.

5.3 CBC vs. HiGHS vs. Gurobi

Solver	License	When to use
CBC (PuLP default)	Open source	Avoid for production; legacy baseline only
HiGHS	Open source, MIT	Default for all new projects; 5–50× faster than CBC
Gurobi / CPLEX	Commercial	When HiGHS hits its limits; parallel B&B; large difficult MILPs

```
prob.solve(HiGHS_CMD(msg=0))          # PuLP + HiGHS
import mip; m = mip.Model(solver_name="HiGHS")  # python-mip + HiGHS
```

Committed thresholds: CBC becomes unacceptable at roughly **5,000 binary variables and 20,000 constraints**. HiGHS carries you to roughly **50,000 binaries / 200,000 constraints**. Above that, buy Gurobi.

5.4 CP-SAT Parallelism

```
solver.parameters.num_workers = 8  # set to physical core count
```

CP-SAT runs a *portfolio* of differently-configured search strategies that share learned clauses — not partitioned search. Speedup is sublinear: 8 workers captures most of the benefit; past 16, returns diminish. For reproducible results set `num_workers = 1`.

5.5 When Bayesian Optimization Degrades

Failure mode	Fix
> 15–20 parameters	Switch to optuna (TPE) or TuRBO
High noise	Model noise explicitly; report posterior-mean best, not best observed
> 1,000 evaluations	Switch to TPE or random search
Cheap evaluations	Use scipy + random restarts first

Committed threshold: GP+EI remains reliably better than random search up to **~20 parameters**. Below 10, the advantage is large and consistent; above 20, use TPE.

---

6 Side-by-Side Comparison

Optimization solvers (find the best value):

Solver class	Objective type	Variables	Optimal guaranteed	Typical scale	Python library
LP	Linear	Continuous	Yes (global)	Millions	PuLP, HiGHS
Network flow	Linear	Continuous (integral by TU)	Yes (global, polynomial)	Millions of edges	networkx
MILP/ILP	Linear	Integer/mixed	Yes (if solved)	Hundreds–thousands	PuLP, python-mip
CP	Constraints only	Discrete	Yes (feasible/infeasible)	Thousands	OR-Tools CP-SAT
Scheduling	Time-indexed	Integer	Yes (makespan)	Jobs × machines	OR-Tools CP-SAT
Routing (VRP)	Distance/time	Integer	Yes (routes)	≤ 1000 stops	OR-Tools routing

Solver class	Objective type	Variables	Optimal guaranteed	Typical scale	Python library
QP (convex)	Quadratic	Continuous	Yes (global)	Thousands	cvxpy
NLP (nonlinear)	Any formula	Continuous	Local only (SLSQP) / heuristic (`diff_evol`)	Tens–hundreds	scipy.optimize
Multi-objective	Multiple	Any	Pareto-optimal	2–20 objectives	pymoo
Stochastic LP	Expected value	Continuous	Yes (expected)	Scenarios × vars	Pyomo / mpi-sppy
Game theory	Per-player payoff	Strategies	Nash / SPNE	2 players, ≥ 2 strategies	nashpy, Gambit
Black-box BO	Simulation	Continuous	Near-optimal	≤ 20 parameters	scikit-optimize, optuna

Non-optimization solvers (answer, verify, or classify):

Solver class	Answer type	Certified	Typical scale	Python library
SQL / database	Query result	Yes	Billions of rows	sqlite3, DuckDB
SAT / SMT	SAT / UNSAT	Yes	Hundreds of rules	Z3
Symbolic math	Closed-form expression	Yes	Single formulas	SymPy, SageMath
Formal proof	Proof term	Yes (kernel-verified)	Theorem + proof	Lean 4
Data validation	Pass / fail + error list	Yes	Millions of rows	pandera, GE
Geospatial	Geometric fact	Yes (float precision)	Thousands of shapes	Shapely

7 Common Mistakes and How to Avoid Them

M1 — Using the wrong solver class

Mistake: using a heuristic (genetic algorithm) when the problem is LP.
Cost: 100× slower, no optimality guarantee.
Fix: if linear and continuous, use LP first.

M2 — Forgetting integrality

Mistake: declaring "number of workers" as continuous.
Cost: LP returns `3.7 workers` — not actionable.
Fix: use `cat='Integer'` in PuLP.

M3 — Conflating infeasible and unbounded

Fix: always check `LpStatus[prob.status]`: `"Optimal"` / `"Infeasible"` (relax a constraint) / `"Unbounded"` (forgot a bound).

M4 — Trusting LLM solver output without verification

Mistake: accepting an LLM's answer to "solve this LP" at face value.
Cost: LLMs make formulation errors at scale — error rates rise sharply as problem size grows.
Fix: always run the actual solver and verify its output. The LLM translates NL → formulation; the solver certifies the answer. See also §4.15 (Formulation Risk).

M5 — Using black-box BO when the formula is known

Mistake: using optuna/GP for a problem with a known analytical gradient.
Cost: 10–100× more evaluations than SLSQP.
Fix: try `scipy.minimize` first.

M6 — Confusing Nash with cooperative optimum

Mistake: recommending the jointly optimal strategy when agents are non-cooperative.
Fix: Nash is what rational agents *do*; cooperative optima require enforceable agreements.

M7 — Forcing MILP on disjunctive constraints (use CP-SAT instead)

Mistake: encoding “no-overlap”, “all-different”, or “if A then not B” with big-M binary constraints in MILP.
Cost: LP relaxation is extremely weak; problems that CP-SAT solves in seconds become hour-long MILP runs.
Fix: if constraints are disjunctive (jobs can’t overlap, employees can’t share a shift), use OR-Tools CP-SAT with `AddNoOverlap`, `AddAllDifferent`, or `AddCircuit`.

M8 — Using MILP when the matrix is totally unimodular

Mistake: modeling transportation, assignment, or bipartite matching as MILP with binary variables.
Cost: paying NP-hard branch-and-bound prices for a problem LP already solves integrally in polynomial time.
Fix: if every column of your constraint matrix has at most one +1 and one −1, use `networkx.min_cost_flow()` — 10–100× faster with no integrality gap.

---

8 Domain Diversity Examples

Domain	Problem	Solver class
Manufacturing	Steel slab sizing: cut master coils into customer-order widths, minimize scrap and changeover setups	MILP (cutting stock / column generation)
Manufacturing	Semiconductor fab scheduling: route wafer lots through photolithography tools with sequence-dependent setup times	CP-SAT (scheduling with interval variables)
Healthcare	Regional vaccine allocation: distribute doses across rural clinics under cold-chain constraints and uncertain demand across scenarios	Two-stage stochastic MILP

Domain	Problem	Solver class
Healthcare	Hospital OR staffing: assign surgical teams to operating rooms, avoid back-to-back shift violations, honor surgeon preferences	CP-SAT
Agriculture	Basin water right allocation: distribute seasonal canal water among farmers under variable inflow and seniority rules	LP / stochastic LP
Agriculture	Crop rotation planning: maximize 5-year yield while meeting soil nitrogen constraints, diversity rules, subsidy eligibility	MILP
Government	Emergency ambulance base location: site 12 bases among 60 candidate locations to maximize population within 8-minute response	MILP (maximal covering location problem)

Silent failure example — ambulance base location: the standard MCLP formulation assumes the nearest ambulance is *always available*. In reality, ambulances are busy 20–40% of the time. The solver returns "94% coverage" — certified "Optimal" — but it answers the wrong question. Fix: use Daskin's MEXCLP, which weights coverage by the probability that at least one nearby unit is free.

Non-Western framing changing the solver class — irrigation water:

- **Western (prior appropriation):** water is a volumetric entitlement with seniority ordering. Model as LP/MILP maximizing basin-wide economic value.
- **East Asian / warabandi (Punjab, Bali *subak*, Iranian *qanat*):** water is allocated by *time slot in a repeating rotation*, proportional to landholding. Decision variables are positions and durations in a cyclic schedule, with no-overlap constraints on a shared channel. This is a **cyclic scheduling problem → CP-**

SAT with `AddNoOverlap`, not a volumetric LP. Same physical resource, same stated goal, structurally different solver class — because the institution governing the resource defines what the decision variable *is*.

---

9 Quick Solver Selector — Natural Language Triggers

NL trigger	Likely solver class
"minimize cost / maximize profit", all formulas linear	LP (PuLP / HiGHS)
"transport", "ship goods from sources to destinations", "flow"	Network flow (networkx)
"how many of each", "yes/no decision", "assign"	MILP/ILP (PuLP)
"schedule", "no two at the same time", "shift", "no-overlap"	CP-SAT (OR-Tools)
"route", "visit all", "vehicle", "depot"	VRP/TSP (OR-Tools routing)
"portfolio", "risk-return", "efficient frontier"	Convex QP (cvxpy)
"revenue curve", "demand elasticity", "non-linear"	NLP (scipy.optimize)
"two objectives", "tradeoff", "Pareto"	Multi-objective (pymoo)
"uncertain demand", "scenarios", "robust"	Stochastic LP (Pyomo)
"both companies", "compete", "equilibrium"	Game theory (nashpy)
"A/B test", "hyperparameter", "limited budget of trials"	Bayesian BO (scikit-optimize / optuna)
"eligibility rules", "if-then policy", "compliance"	SMT (Z3)
"data quality", "null check", "column range", "validate"	Data validation (pandera / GE)
"service area", "polygon overlap", "coverage", "distance"	Geospatial (Shapely)
"find all", "filter by", "count where", "join tables"	SQL / database engine
"prove", "simplify", "closed form", "symbolic", "integrate"	Symbolic math (SymPy / SageMath)

NL trigger	Likely solver class
“formally verify”, “kernel-checked proof”, “theorem”	Formal proof (Lean 4)

---

10 The Verification Ladder

A solver answer is only as trustworthy as the check you apply to it.

> **Critical warning:** C1–C6 all verify the solution against the model you wrote. **None verifies the model against the real world.** The empirically dominant failure mode in applied optimization is a wrong or incomplete formulation — and the entire ladder is blind to it. See §4.15. Always run C0 before C1.

ID	Class	Verification method	What it proves	What it still misses
C0	Model validation	Degenerate test + shadow-price inspection + expert review of formulation	Model represents the real problem	Still relies on expert judgement
C1	Categorical oracle	Solver status == Optimal / SAT	Solution exists and is feasible for the model	Correctly solves the *wrong* model
C2	Numeric round-trip	Substitute solution back, recompute independently	Objective value is arithmetically correct	Formulation error shared by calculation and model
C3	Independent recomputation	Solve same problem a second way (different library)	No implementation/library bug	Both implementations share the same conceptual error

ID	Class	Verification method	What it proves	What it still misses
C4	Structural diff	Compare solution structure against known shape	Structural correctness	Structurally plausible answer can still be economically wrong
C5	Two independent solvers	Two solvers agree	High confidence in computation	**Both share the formulation** — failure is correlated while feeling like independent verification
C6	Falsification-first	Property-based test: random inputs, assert no counterex-ample	Robustness / generaliza-tion	Misses adversarial edge cases outside the generator's distribution
C7	Kernel-checked proof	Formal proof verified by Lean 4 / Coq	Mathematical certainty	Proves the formal statement — not that it represents reality

Ranking by false confidence produced (most dangerous first): C5 > C1 > C2 > C3 > C6 > C4 > C7.

- **C5 is most dangerous by false-confidence ratio** — two solvers agreeing feels like strong independent corroboration while sharing the single most likely point of failure: the formulation.
- **C1 is most dangerous by volume** — it is the check everyone runs, and the word "Optimal" actively misleads in ordinary English.

Special cases: - **Non-convex NLP / MINLP:** no C-class below C7 certifies global optimality. `differential_evolution` provides no optimality certificate. - **Bayesian**

optimization: no known optimum to compare against. Verification means surrogate calibration, convergence diagnostics, restart agreement, and re-evaluating the recommended point to rule out noise artifacts.

---

11 Worked Examples by Solver Class

Solver class	Problem	Variable / Objective / Constraint	Key Python invocation
LP	A refinery blends 3 crude streams into gasoline and diesel, each with different octane, sulfur, and cost. Meet product specs and contracted volumes; minimize total crude cost.	`x[crude, product] ≥ 0` (continuous barrels). Obj: `min Σ cost·x`. Blending specs in linear form; reformer capacity; volume balance.	`prob.solve(HiGHS_CMD(msg=0))`
MILP	A hospital staffs 6 ORs × 3 shifts with 40 nurses. Each nurse has certified specialties, a 4-shift/week cap, and a wage. Opening an OR for a shift incurs a fixed setup cost. Minimize total cost.	`y[n,o,s] ∈ {0,1}`, `z[o,s] ∈ {0,1}`. Obj: `min Σ wage·y + Σ fixed·z`. Coverage per specialty; `y ≤ z` (linking); shift cap.	`prob.solve(HiGHS_CMD(timeLimit=3 gapRel=0.01))`

Solver class	Problem	Variable / Objective / Constraint	Key Python invocation
CP	A call center schedules 15 agents over 21 shifts (7 days × 3). Each shift needs exactly 3 agents; no agent works night-then-morning; each works 5 shifts with ≥ 2 consecutive days off; two agents must never share a shift. Find any feasible roster.	`x[a,d,s] ∈ {0,1}` BoolVar. `AddExactlyOne`, forbidden transitions, `AddAtMostOne` for conflict pair.	`solver.parameters.max_time_in_se = 60; solver.Solve(model)`
Stochastic LP	A vaccine distributor orders doses now (€8/dose). After demand is observed across 12 scenarios, unmet demand is covered by air freight (€22/dose); surplus is wasted (€3/dose). Minimize expected total cost.	Stage 1: `q ≥ 0`. Stage 2: `short[ω], surplus[ω] ≥ 0`. Obj: `min 8q + Σ p[ω]·(22·short[ω] + 3·surplus[ω])`. Non-anticipativity: `q` has no ω index.	`SolverFactory("highs").solve(m)` (Pyomo)

Solver class	Problem	Variable / Objective / Constraint	Key Python invocation
Game theory	Two grocery chains simultaneously choose to enter or stay out of a new suburb. Entry costs €4M; sole entrant earns €10M; both enter → €1M each; staying out earns €0. Find all Nash equilibria.	2×2 payoff matrices A, B. Equilibria: two asymmetric pure + one mixed.	`list(nash.Game(A,B).support_enum`
Bayesian BO	A materials lab tunes a perovskite solar cell: 6 continuous parameters (anneal temp, time, precursor ratio, spin speed, humidity, additive %). Each fabrication cycle takes 8 hours. Budget: 40 experiments. Maximize PCE.	6 continuous, box-bounded. Black-box noisy `f(x) → PCE`. GP Matérn-5/2 surrogate; log-EI acquisition.	`gp_minimize(objective, dimensions, n_calls=40, n_initial_points=12, noise="gaussian")`

12 End-to-End Worked Example

Problem (NL): “We manage 3 delivery zones in a city. The zones overlap in some areas, leaving some customers unassigned. We want to know: (a) how much overlap exists, (b) which customers are unassigned, (c) whether total coverage exceeds 90%.”

Solver selection walk-through:

1. Does it involve continuous variables and a linear objective? **No** — it's about geometry.
2. Does it involve scheduling or routing? **No** — static zones.
3. Does it involve polygon shapes and geographic regions? **Yes → Shapely**.

Formulation: - Decision variables: none (analysis problem, not optimization) - Key operations: `zone_a.intersection(zone_b)` for overlap; `unary_union([...]).difference(service_area` for gap; `zone.contains(point)` for assignment - Verification: `coverage_ratio >= 0.9` (C2 — recompute area independently)

Result: overlap = 6 km², gap = 4 km², 2 customers unassigned, coverage = 95% → verification passes (C2).

---

13 Resources

Python libraries (all open-source)

Library	Install	Docs
PuLP	`pip install pulp`	coin-or.github.io/pulp
HiGHS	bundled with PuLP or `pip install highspy`	highs.dev
OR-Tools	`pip install ortools`	developers.google.com/optimization
cvxpy	`pip install cvxpy`	cvxpy.org
scipy.optimize	`pip install scipy`	docs.scipy.org
pymoo	`pip install pymoo`	pymoo.org
nashpy	`pip install nashpy`	nashpy.readthedocs.io
Gambit (3+ player games)	see gambit-project.org	gambit-project.org
scikit-optimize	`pip install scikit-optimize`	scikit-optimize.github.io
optuna	`pip install optuna`	optuna.readthedocs.io
z3-solver	`pip install z3-solver`	github.com/Z3Prover/z3
pandera	`pip install pandera`	pandera.readthedocs.io
great-expectations	`pip install great-expectations`	greatexpectations.io
Shapely	`pip install shapely`	shapely.readthedocs.io
python-mip	`pip install mip`	python-mip.com
networkx	`pip install networkx`	networkx.org

Library	Install	Docs
Pyomo + mpi-sppy	`pip install pyomo mpi-sppy`	pyomo.org

Books, courses, and videos

Books - *Introduction to Operations Research* (Hillier & Lieberman) — classic textbook; covers LP, MILP, scheduling, networks from first principles - *The MILP Optimization Handbook* (DeJans, 2024) — hands-on practitioner guide; builds LP → MIP progressively with Python examples

Videos - MIT OpenCourseWare — Introduction to Mathematical Programming (15.053) — full lecture videos free; LP, MILP, network flows, integer programming with business applications - Discrete Optimization MOOC — University of Melbourne (Coursera) — Prof. Peter Stuckey; LP, MILP, constraint programming, and local search side-by-side; audit free

Articles - MIP Solvers Unleashed — Medium - Comprehensive Guide to MILP Modeling Techniques — TDS - An Overview of Math Programming Solvers — GAMS Blog - NL4Opt Competition — NL → optimization problem formulation - Operational Research: Methods and Applications — arXiv survey

---

Appendix — Glossary

Term	Meaning
Feasible	A solution that satisfies all constraints
Infeasible	No solution exists that satisfies all constraints
Unbounded	The objective can be made arbitrarily good (usually means a constraint is missing)
Global optimum	The best solution across the entire feasible space
Local optimum	Best in a neighborhood; may not be globally best (nonlinear problems)
Pareto front	The set of non-dominated solutions in a multi-objective problem

Term	Meaning
Dominated	Solution A dominates B if A is at least as good on all objectives and strictly better on at least one
Nash equilibrium	Each player's strategy is the best response to the others'; no player gains by deviating unilaterally
SPNE	Subgame-perfect Nash equilibrium — Nash equilibrium requiring rational play in every sub-game
Branch-and-bound	MILP algorithm: solve LP relaxation, branch on a fractional variable, prune branches that cannot improve the best known integer solution
Totally unimodular (TU)	A matrix where every square submatrix has determinant 0, +1, or −1; the LP relaxation of a TU-structured problem is automatically integer-valued
Shadow price	The marginal improvement in the objective from relaxing a constraint by one unit; zero for non-binding constraints
Validity range	The range over which a shadow price remains valid before another constraint becomes binding
Formulation risk	The gap between the practitioner's real-world intent and the mathematical model actually solved
Dominance resistance	In many-objective optimization: when nearly all solutions are mutually non-dominated, collapsing selection pressure in evolutionary algorithms
Reference directions	In NSGA-III: pre-specified vectors on the unit simplex that enforce population diversity in many-objective optimization
Acquisition function	In Bayesian optimization: a function (e.g., Expected Improvement) that scores candidate points by balancing exploration and exploitation

Term	Meaning
Surrogate model	In Bayesian optimization: a probabilistic model (e.g., Gaussian Process) approximating the expensive objective function
C0–C7	Verification class taxonomy (model validation through kernel-checked proof) — see §10

---

v0.2 — Synthesized from five model reviews: GPT-4o/o3, Claude Opus 5, Gemini 2.5 Pro, GLM-4-7 (Zhipu AI), Qwen-3-7-plus (Alibaba). Open questions from reviewer disagreements are tracked in `solver-research-plan.md §A.2`.